

บทที่ 4

การออกแบบและพัฒนา Software

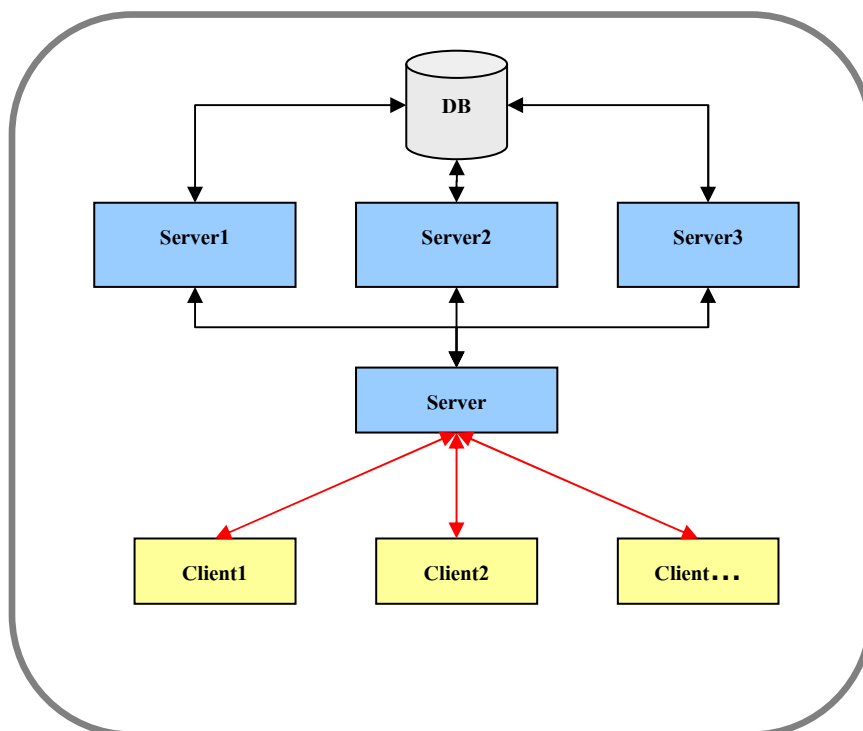
เนื้อหาภายในบทนี้จะทำการพูดถึง ระบบการทำงานของ Software ทั้งหมดที่มีภายในโครงงาน โดยภายในโครงงานนี้ได้ใช้ภาษา C++ ในการพัฒนาทั้งหมด และใช้ API DirectX มาช่วยในการเขียนโปรแกรม ในส่วนของ Programming สามารถแบ่งเนื้อหาออกเป็นส่วนต่างๆ ได้หลายส่วน ดังนี้

4.1 เครื่องมือที่ใช้ในการพัฒนาโปรแกรม

- Visual Studio 2003
- DirectX SDK 9.0 เป็น API

4.2 ส่วนการทำงานของ Software

การทำงานของ Software นั้นแบ่งออกเป็นหลายส่วนด้วยกัน ดังรูปต่อไปนี้



รูปที่ 22 แสดงโครงสร้างของระบบ

4.2.1 ส่วนโปรแกรม Client

เป็นโปรแกรมที่ใช้สำหรับติดตั้งบนเครื่อง Client เพื่อเข้าทำการเล่นเกม รายละเอียดของตัวเกมรวมไปถึงการทำงานและการแสดงผลทั้งหมดจะถูกควบคุมด้วยโปรแกรมชุดนี้ ผู้เล่นจำเป็นต้องควบคุมตัวละครผ่านโปรแกรมชุดนี้ เมื่อโปรแกรมเริ่มทำงานโปรแกรมชุดนี้จะทำการติดต่อไปยัง Program Server

4.2.2 ส่วนโปรแกรม Server Load Balance

เป็นโปรแกรมที่ใช้สำหรับแยก Connection ต่างๆให้ติดต่อกับ Server ภายในอย่างถูกต้อง และมีจำนวน Connection ใกล้เคียงกัน โดยสังเกตจาก Map ของผู้เล่นและข้อมูลใน Database ว่าขณะนั้นมีจำนวน Connection ใน Server เท่าใด

4.2.3 ส่วนโปรแกรม Server Game1,2,3

เป็นโปรแกรมที่ทำงานคล้ายกันต่างกันที่แต่ละ Server จะจัดการกับผู้เล่นคนละ Map และทำการส่งข้อมูลเฉพาะ Map ให้กับผู้เล่นเท่านั้น

4.2.4 ส่วนโปรแกรม Database

เป็นโปรแกรมที่ใช้ในการควบคุมการทำงานต่างบนฝั่ง Database โดยโปรแกรมชุดนี้หลักๆแล้วจะทำการควบคุมตัวศัตรูทั้งหมดภายในเกมให้เคลื่อนไหวเข้ามาต่อสู้กับผู้เล่นรวมไปถึงควบคุมการเคลื่อนไหวต่างๆของตัวศัตรูอีกด้วย

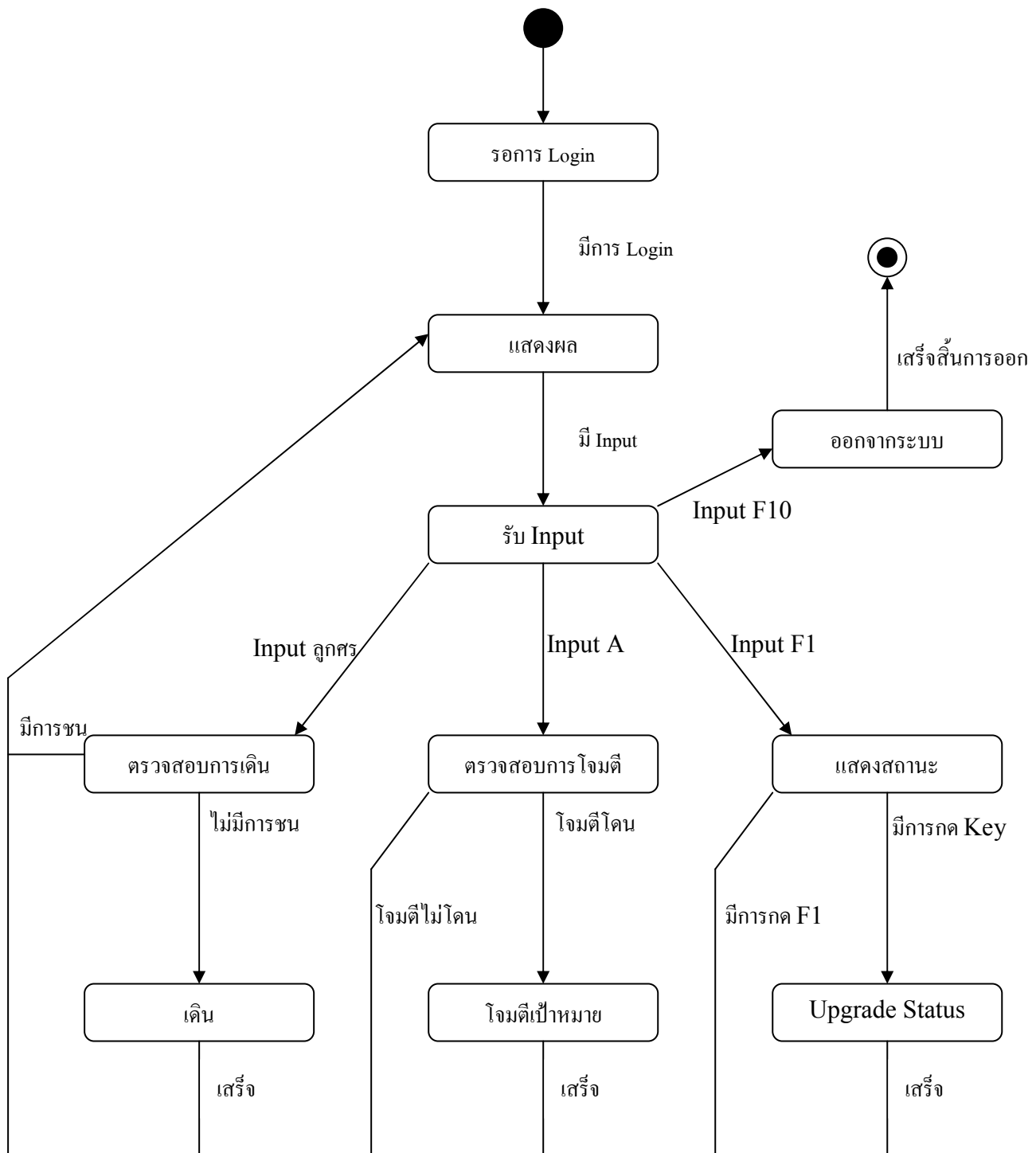
4.3 การทำงานของ Game (Client Programming)

เป้าหมายในการพัฒนา

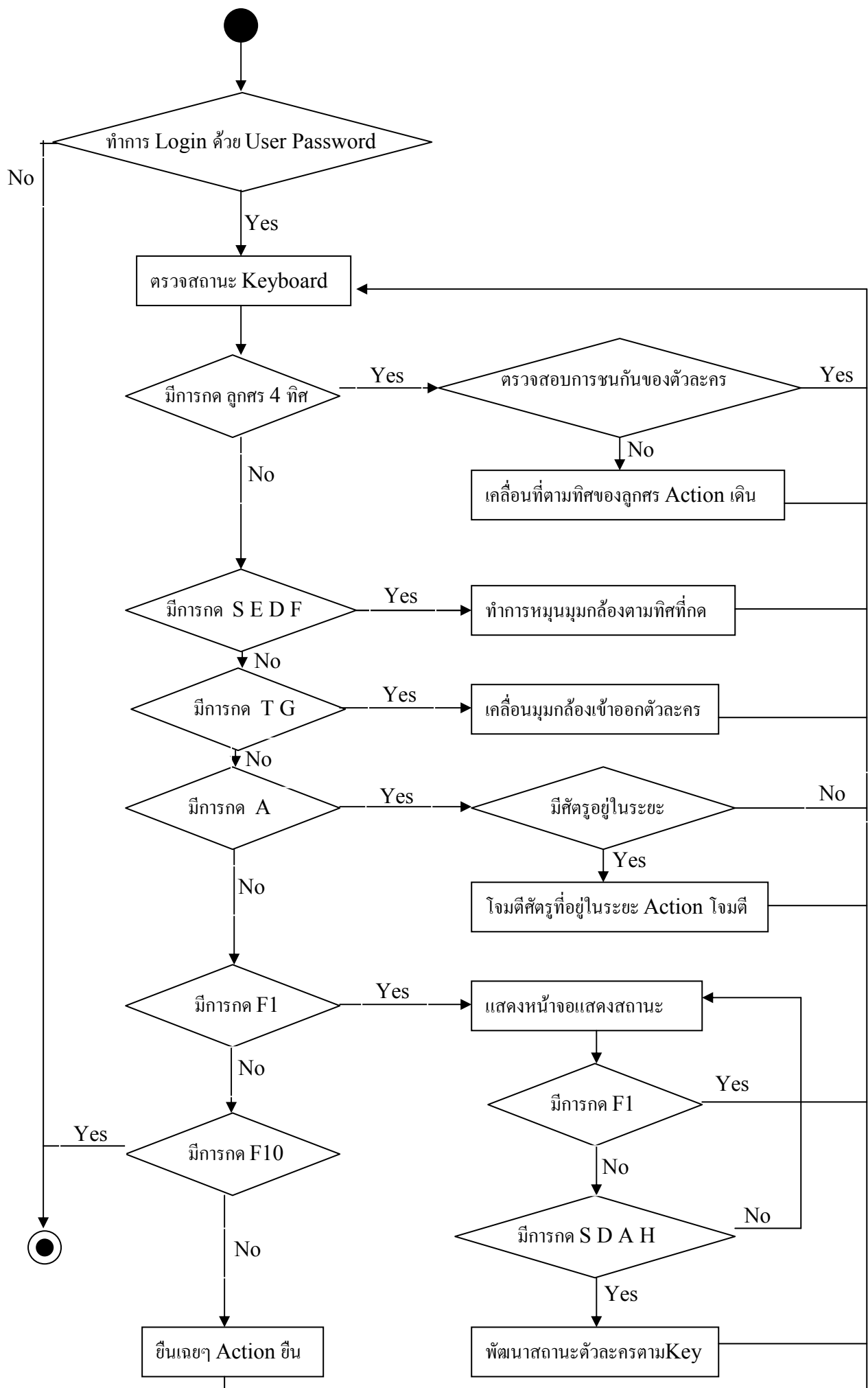
1. พัฒนาโปรแกรม Client ที่สามารถแสดงภาพสามมิติรวมไปถึงแสดงการเคลื่อนไหว (Frame Animation) ได้
2. โปรแกรมสามารถรับ Input ผ่าน Key Board และนำมาใช้ในการควบคุมการทำงานของโปรแกรมได้
3. พัฒนาโปรแกรมให้สามารถรองรับผู้เล่นจำนวนมากได้โดยทำการติดต่อกันผ่าน Server
4. พัฒนาโปรแกรมให้สามารถเล่นเป็นเกมแนวผจญภัยสามมิติและผู้เล่นสามารถเล่นได้อย่างสนุกสนาน

พัฒนาการของโปรแกรม

1. เขียนโปรแกรมเพื่อสร้าง Window ที่รองรับการเรนเดอร์ขึ้น สามมิติ
2. เขียนฟังก์ชันรับข้อมูลจาก X file เพื่อนำข้อมูล Mesh Object มาใช้
3. เขียน Class เพื่อใช้ในการควบคุมการทำงานของ Mesh เช่น
 - แสดงผลโมเดลสามมิติ (Render Mesh)
 - ควบคุม Position ของ Mesh (Translation Mesh)
 - การหมุน Mesh (Rotation)
 - การกำหนดขนาดย่อขยาย Mesh (Scaling)
4. เขียนโปรแกรมโดยใช้ Direct Input เพื่อรับข้อมูลจาก Key Board เพื่อควบคุมตัวละคร
5. เขียนโปรแกรมควบคุมมุมมองกล้องภายในโลกสามมิติโดยสามารถควบคุมได้ดังต่อไปนี้
 - หมุนมุมมองกล้องไปทางซ้ายและขวาของตัวละคร
 - หมุนมุมมองกล้องไปทางด้านบนและล่างของตัวละคร
 - Zoom เข้าและออกจากตัวละคร
6. เขียนโปรแกรมเพื่อรองรับการทำงานของ Frame Animation รวมไปถึงกำหนดการเคลื่อนไหวของ Object ให้สอดคล้องกับ Input ที่ได้รับมา
7. นำการทำงานทั้งหมดในข้างต้นมาประยุกต์เขียนโปรแกรมเกมรวมไปถึงการควบคุมการทำงานส่วนต่างๆ ของเกม ดังนี้
 - แสดงตัวละครพร้อมการเคลื่อนไหวให้สอดคล้องกับ Input ที่ได้รับ
 - มีการตรวจสอบการชนกันของตัวละคร โดยทำการสร้างกล่องครอบตัวละครไว้เพื่อใช้ในการตรวจสอบ
 - ตรวจสอบการโจมตีโดยหากมีการต่อสู้กันเกิดขึ้นจะต้องมีการลดเลือดกัน
 - มีการกำหนดค่าสถานะ (Status) ต่างๆ ภายในเกมให้กับตัวละครต่างๆ
8. เขียนโปรแกรมเพื่อแสดงสถานะต่างๆ ผ่าน Interface โดยใช้การแสดงผลสองมิติ
9. ทำการเพิ่มเสียงเพลง รวมไปถึงเสียงต่างๆ ของตัวละครภายในเกม โดย Direct Sound
10. พัฒนาโปรแกรมติดต่อรับ-ส่งข้อมูลระหว่าง Client-Server เพื่อให้การเคลื่อนไหวและการทำงานของตัวละครถูกรายงานผลไปยัง Server ในขณะเดียวกัน Client ก็สามารถรับข้อมูลของผู้เล่นคนอื่นๆ รวมไปถึงศัตรูได้
11. ทำการตรวจสอบ และแก้ไขข้อผิดพลาดที่พบ พร้อมกับพัฒนาในส่วนของความสวยงาม



รูปที่ 23 State Diagram ของ Software Client

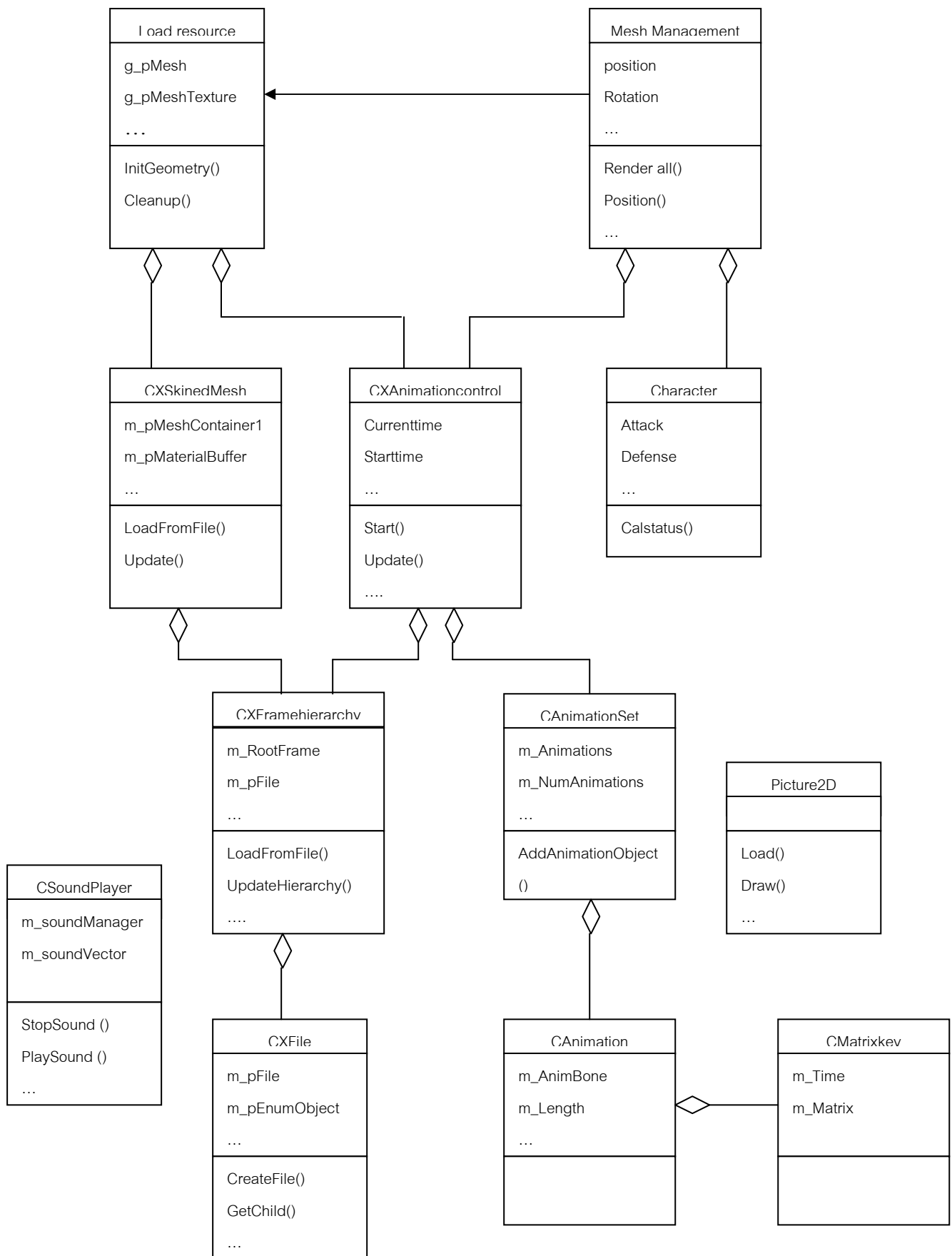


รูปที่ 24 Flow Chart แสดงถึงการทำงานหลักๆของ Program Client

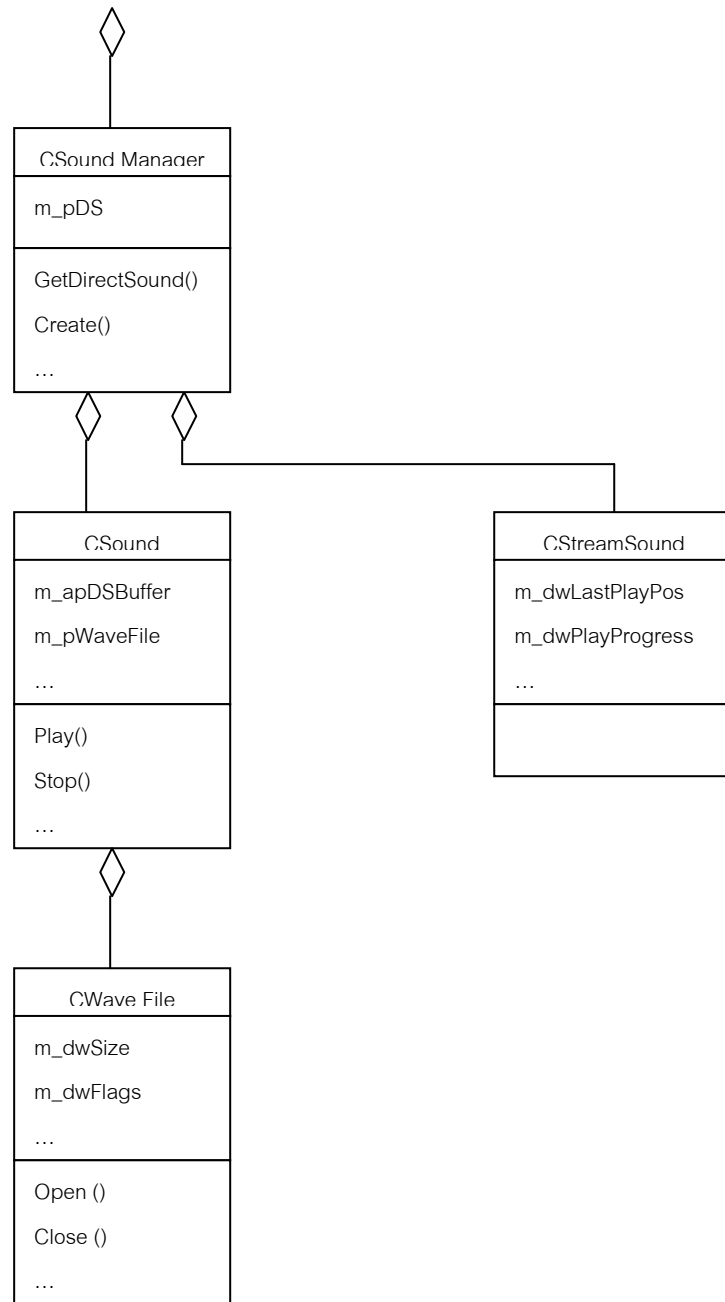
Class ที่ใช้ใน Program Client

ในโปรแกรมนั้นมีการสร้าง Class และนำมาใช้อยู่หลาย Class ด้วยกัน โดยมีรายละเอียดดังต่อไปนี้

1. Load Resource Class ใช้ในการเก็บข้อมูลเกี่ยวกับตัวโมเดลสามมิติต่างๆ รวมไปถึงการทำ Animation อีกด้วย
2. Mesh Management Class เป็น Class ที่ใช้ในการควบคุม object ที่จะทำการแสดงผลสามมิติ เช่น แสดงตำแหน่งไหน แสดงFrameใด ขนาดเท่าไร เป็นต้น รวมไปถึงทำการแสดงผลบนหน้าต่างด้วย
3. CXSkinnedMesh Class เป็น Class ที่ใช้ในการจัดเก็บตัว Model
4. CXFramehierarchy Class ใช้ในการจัดการ Frame เก็บ Root Frame
5. CXFile Class เป็นตัวจัดการและติดต่อ DirectX File
6. CXAnimationcontrol Class เป็น Class ทำหน้าที่ในการควบคุม และUpdate Animation ทั้งหมด
7. CAnimationSet Class เป็น class ที่จัดเก็บ Animation ทั้งหมดของ Object 1 Object เอาไว้ ภายใน 1 AnimationSet ประกอบด้วย Animation หลายๆอันด้วยกัน
8. CAnimation Class เป็น Class ที่รวบรวมAnimation ไว้
9. CMatrixkey Class เป็นตัวระบุว่า Animation นี้ควรนำ Matrix ไฉมาใช้
10. Character Class เป็น Class ที่ใช้เก็บสถานะต่างๆของตัวละคร 1 ตัวเช่น ค่าพลังในการโจมตี ตั้งรับ เลือดที่เหลืออยู่ เป็นต้น
11. CSoundPlayer Class ใช้ในการจัดการเสียงตั้งแต่ Load ไฟล์เสียงเข้ามา จนถึงเริ่มเปิดเสียงต่างๆ
12. Picture2D Class ใช้ในการแสดงผลภาพ 2 มิติต่างๆรวมถึง interfaceด้วย

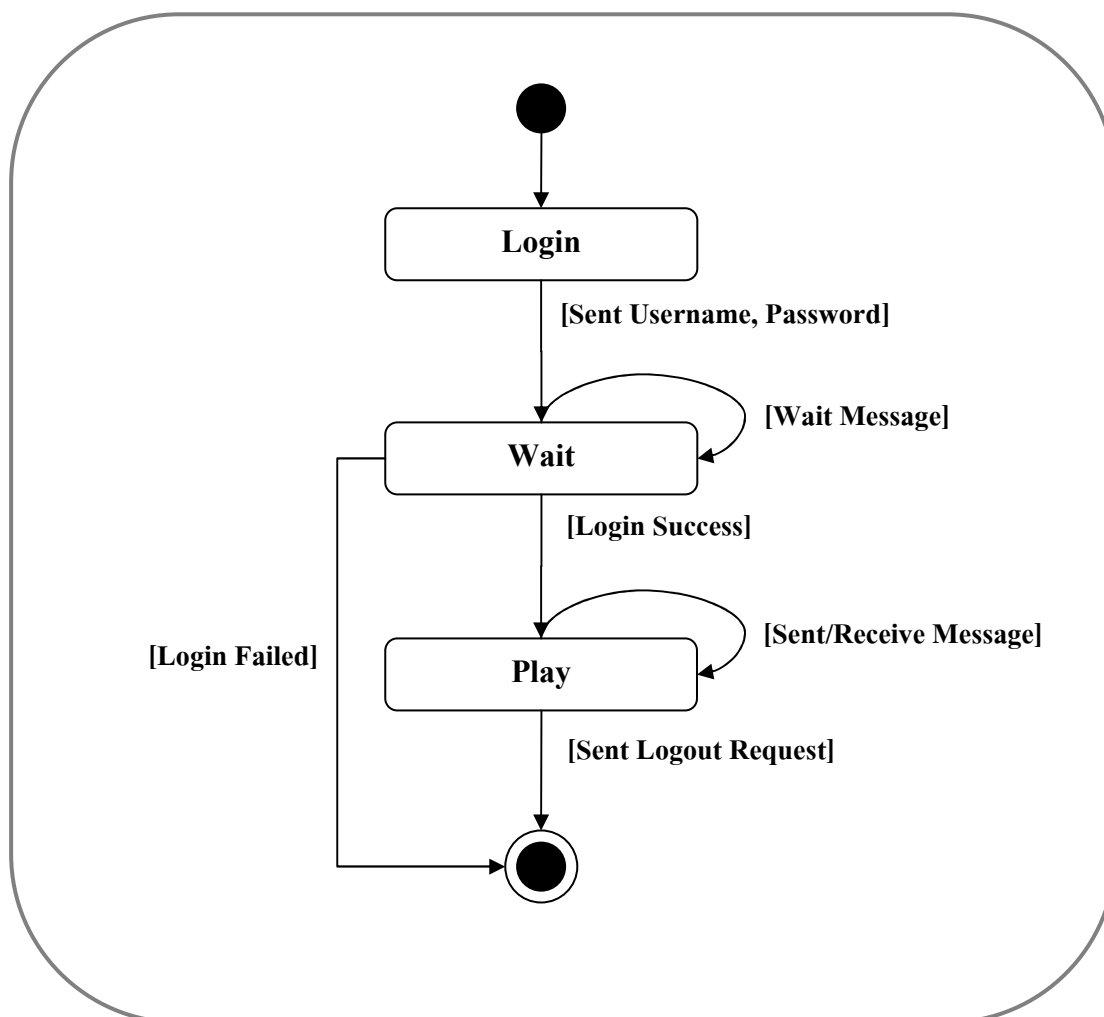


รูปที่ 25 Class Diagram แสดงถึง Class ที่ใช้ในการจัดการที่มูลที่จะใช้ในการแสดงผล



รูปที่ 26 แสดง Class Diagram แสดงถึง Class ที่ใช้ในการจัดการเสียงและแสดงภาพ 2 มิติ

State การทำงานของ Client Application ในแง่การติดต่อกับระบบ



รูปที่ 27 แสดง State Diagram ของ Client Application

จากรูป จะเห็นได้ว่าเริ่มต้นโปรแกรมจะอยู่ในสถานะ 'Login' ซึ่งเป็นสถานะที่รอรับ Username และ Password จากผู้เล่น จากนั้นจะทำการส่ง Username และ Password ที่รับมาไปยัง Server เพื่อทำการตรวจสอบและร้องขอเข้าสู่ระบบ จากนั้นสถานะของ Client จะเปลี่ยนเป็น 'Wait' และรอการตอบรับจาก Server การตอบรับจาก Server มี 2 กรณี ดังนี้

- Message ที่ตอบกลับมา คือ 'Success' Client จะเปลี่ยนสถานะของตนเองเป็น 'Play' และทำการรับ-ส่งข้อมูลกับ Server เข้าเล่นเกมได้
- Message ที่ตอบกลับมาเป็นข้อความอื่นหรือ 'Time out' Client จะทำการ Terminate ตนเองทันที

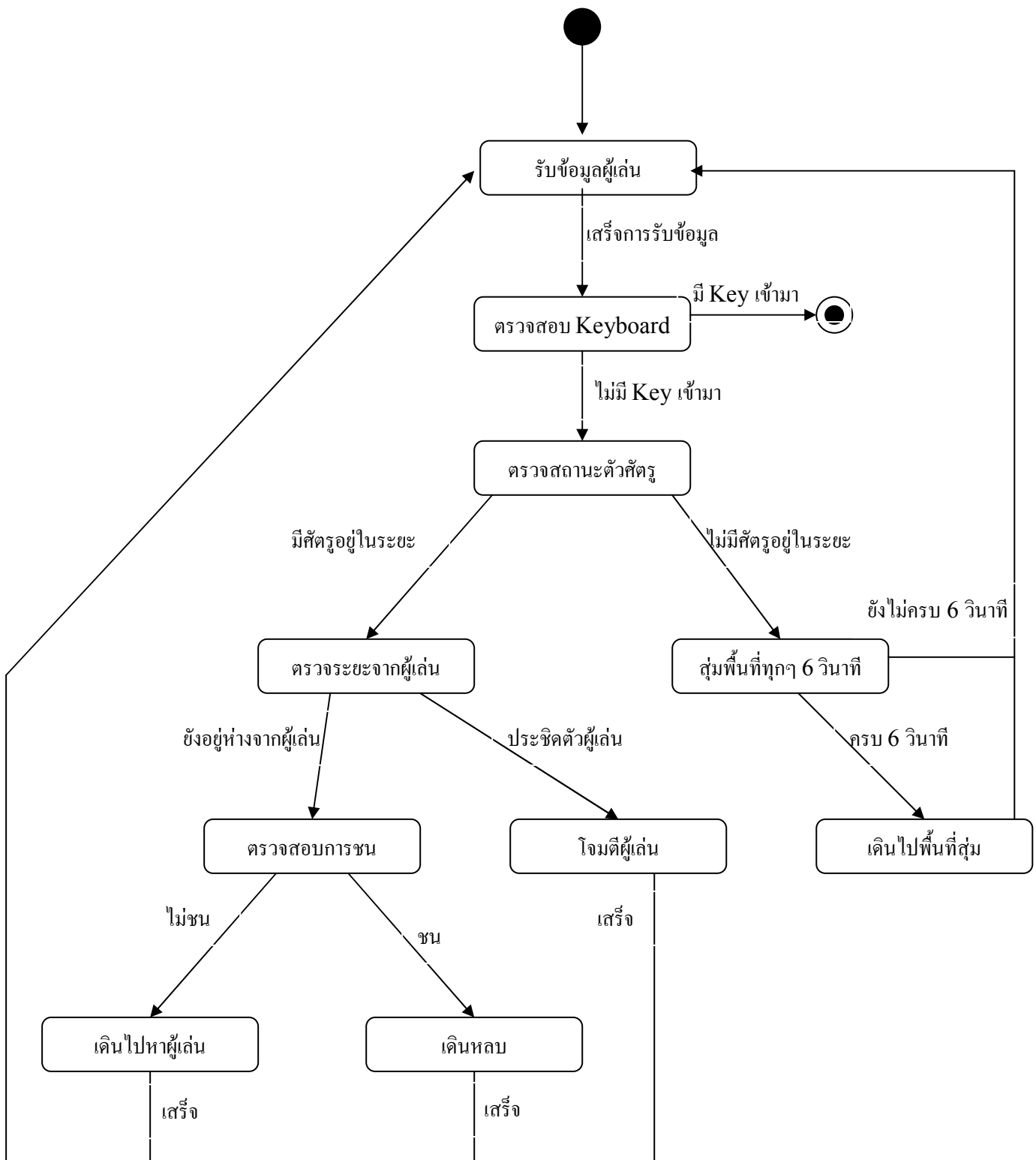
4.4 การทำงานของ Program ควบคุมข้อมูลฝั่ง Database

เป้าหมายในการพัฒนา

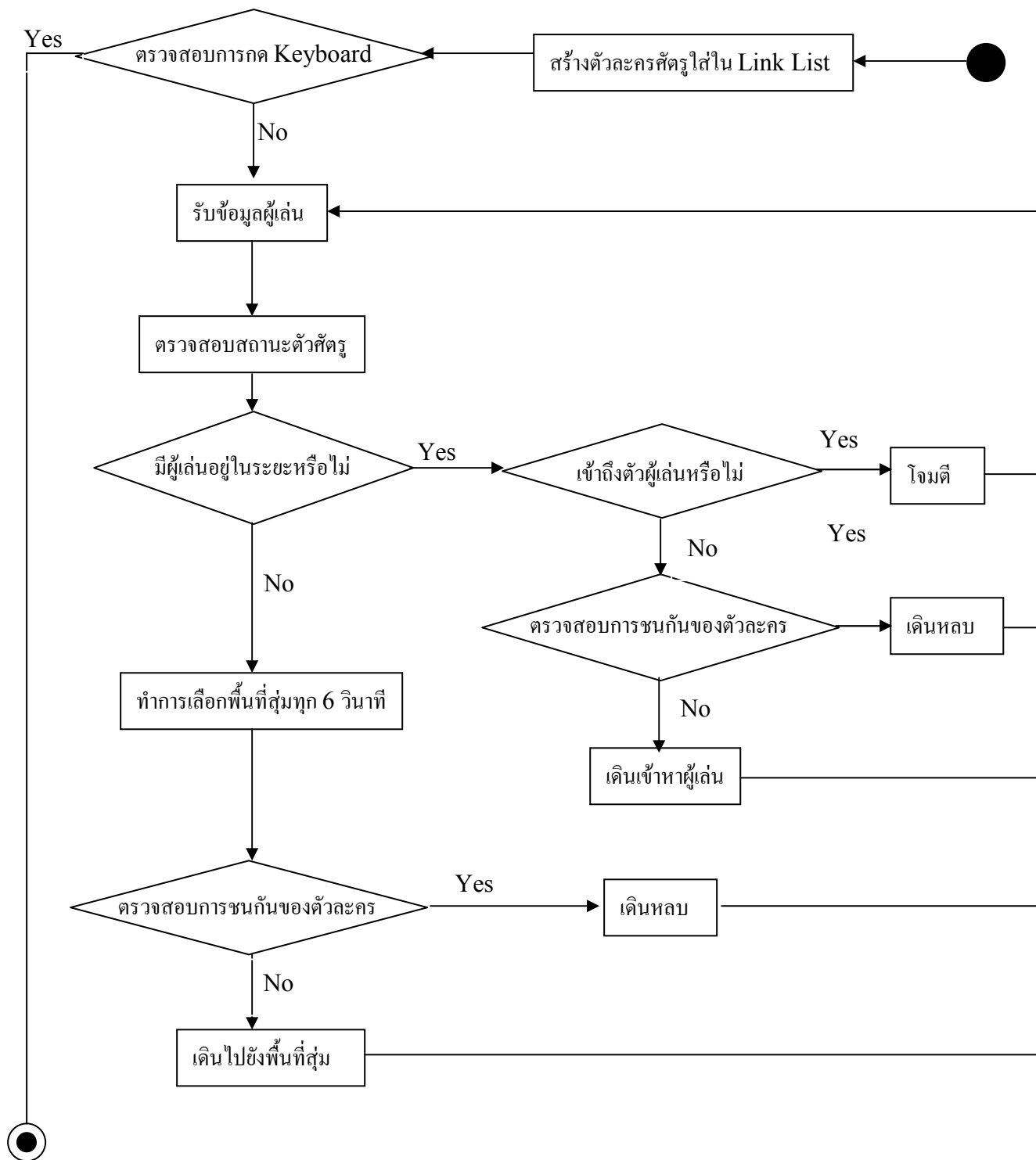
1. โปรแกรมถูกพัฒนาเพื่อใช้ในการควบคุมการทำงานของตัวศัตรูต่างๆภายในเกม ดังนี้
 - ควบคุมการเคลื่อนไหวของตัวศัตรูให้เดินบนพื้นที่สูง
 - ควบคุมการเคลื่อนที่ของตัวศัตรูให้เข้าโจมตีผู้เล่นเมื่อมีผู้เล่นอยู่ในระยะ
 - หากมีการต่อสู้กับผู้เล่นจะมีการคำนวณค่าสถานะต่างๆเพื่อลดเลือดของผู้เล่น
2. โปรแกรมถูกพัฒนาเพื่อใช้ในการกำหนดค่าสถานะรวมไปถึงข้อมูลต่างๆ ของตัวศัตรู

พัฒนาการของโปรแกรม

1. เขียนโปรแกรมที่ใช้ในการติดต่อกับ Database ได้
2. เขียนโปรแกรมเพื่อสร้างตัวละครศัตรูและกำหนดสถานะต่างๆให้กับตัวละครศัตรู
3. เขียนโปรแกรมเพื่อใช้ควบคุมตัวละครศัตรู ดังต่อไปนี้
 - ควบคุมการเคลื่อนไหวพื้นฐาน
 - ตรวจสอบการชนกันของตัวละครแต่ละตัวได้
 - ให้ศัตรูเดินในพื้นที่สูงในขณะที่ไม่ีผู้เล่นอยู่ในระยะ
 - ศัตรูสามารถเดินไปโจมตีผู้เล่นได้เมื่อผู้เล่นอยู่ในระยะ
4. เขียนโปรแกรมเพื่อใช้ติดต่อข้อมูลรับส่งกับ Database เพื่อให้สามารถทราบความเคลื่อนไหวของผู้เล่นแต่ละคนและให้ตัวละครศัตรูตอบโต้กลับไปได้อย่างถูกต้อง



รูปที่ 28 State Diagram ของ Software ที่ใช้ในการควบคุมตัวศัตรูฝั่ง Database



รูปที่ 29 Flow Chart แสดงการทำงานของ Program ที่ใช้ในการควบคุมตัวศัตรู

4.5 การทำงานของ Server Load Balancing (Server Programming)

เป้าหมายในการพัฒนา

1. โปรแกรมถูกพัฒนาขึ้นเพื่อทำการแยกแยะประเภทของผู้เล่นว่าอยู่ในสถานะ (Map) ไตและทำการแบ่งจำนวน Connection ให้แต่ละ Server มีจำนวนใกล้เคียงกัน

พัฒนาการของโปรแกรม

1. ออกแบบ Table ที่ใช้ในการเก็บข้อมูล Connection ที่มีในระบบ
2. เขียนโปรแกรมติดต่อกับ Database
3. พัฒนาโปรแกรมเพื่อทำการ Update ข้อมูลภายใน Table ที่ได้ออกแบบไว้
4. พัฒนาโปรแกรมในส่วน Balance Connection และทำการรับส่งข้อมูลกับ Client

4.6 การทำงานของ Server (Server Programming)

เป้าหมายในการพัฒนา

1. โปรแกรมถูกพัฒนาขึ้นเพื่อทำการรับส่งข้อมูลกับ Client และ Update ข้อมูลลง Database

พัฒนาการของโปรแกรม

1. ออกแบบ Table ที่ใช้ในการเก็บข้อมูลสถานะต่างๆ ของผู้เล่น
2. เขียนโปรแกรมติดต่อกับ Database
3. พัฒนาโปรแกรมให้สามารถติดต่อกันระหว่าง Server และ Client
4. พัฒนาโปรแกรมเพื่อทำการ Update ข้อมูลภายใน Table ที่ได้ออกแบบไว้
5. พัฒนาโปรแกรมให้ Update สถานะของ Server (ตนเอง) เพื่อให้ Server Load Balancing รับรู้ว่า Server ไตยังคงทำงานอยู่